Lecture 6 — Binary Search Trees

A binary search tree is a binary tree in which, for every node, all keys in the left subtree are less than the node key and all keys in the right subtree are greater. This ordering property is what makes search fast.

Search, insert, and delete all descend from the root, comparing against the current node and moving left or right, so their cost is proportional to the height of the tree.

The worst case is a degenerate tree — e.g., inserting keys in sorted order — which degrades to a linked list with height n and O(n) operations. This motivates the balanced trees of the next lecture.

Tree traversals come in three standard orders. Pre-order visits the node, then the left subtree, then the right. In-order visits left, node, right, and on a BST yields the keys in sorted order. Post-order visits children before the parent.

In-order traversal of a BST is the classic O(n) way to output the elements in sorted order without an explicit sort, and it is used by tree-based sorting algorithms such as TreeSort.

Lecture 8 — Balanced Trees

A balanced search tree keeps its height logarithmic in the number of keys so that the O(log n) bound on search, insert, and delete is actually achieved rather than hypothetical.

An AVL tree enforces the invariant that for every node, the heights of its two subtrees differ by at most one. Insertions and deletions that violate this invariant are fixed with single or double rotations.

A red-black tree relaxes the balance requirement slightly, guaranteeing that the longest path from root to leaf is at most twice the shortest. It needs fewer rotations during insertion and is used by many standard libraries.

The height of a red-black tree with n nodes is at most 2*log2(n+1), which is why the operations are O(log n) in the worst case while still allowing fast updates.

Self-balancing trees are the workhorse behind ordered maps and ordered sets: membership queries, predecessor and successor queries, and range scans are all supported in logarithmic time.

Lecture 11 — Graph Traversals: BFS and DFS

A graph is a set of vertices connected by edges, and it can be directed or undirected. Two common representations are the adjacency list, where each vertex stores its neighbors, and the adjacency matrix, a V-by-V boolean grid.

Adjacency lists use O(V+E) space and make iterating over a vertexs neighbors cheap, which most graph algorithms require, so lists are usually the right default. Matrices use O(V^2) space but offer O(1) edge-existence checks.

Breadth-first search (BFS) explores the graph in layers, using a queue, and computes the shortest path in terms of the number of edges from a source vertex. It is the algorithm behind pathfinding in unweighted grids.

Depth-first search (DFS) explores as far as possible along each branch before backtracking, using a stack or recursion. DFS is the basis for topological sorting, cycle detection, and connected-component analysis.

BFS has the useful property that when it first reaches a vertex, the route taken is a shortest path. DFS instead prioritizes depth, which makes it natural for exploring all reachable structure but not for finding the shortest route.